

Consultant Cybersécurité

Analyse de Malwares utilisant le Machine Learning

NOM Prénom
GOHAUT Renan

Mastère Spécialisé :
Expert Forensic et Cybersécurité

Responsable Pédagogique
ELGALAI Reza

Année Scolaire
2025 - 2026

Résumé (150 mots)

Ce stage s'est déroulé dans l'entreprise Aperture Science, au sein du service Recherche et Développement. Il a consisté à travailler en coopération avec un groupe d'ingénieur sur un dispositif à effet tunnel quantique, dans le but de fournir, aux cobayes, un outil leur permettant de placer deux portails où des objets peuvent passer au travers.

Les différentes phases de travail successivement réalisées sont :

- Déterminer les besoins et les technologies à utiliser
- Réaliser des tests en laboratoire afin de confirmer l'efficacité des portails
- Observer le comportement d'un cobaye lors de l'utilisation du ASHPoD

L'étude s'appuie sur des technologies de pointes et l'expérimentation quantique.

L'enjeu est une amélioration du déplacement de Sapiens en lui permettant d'appréhender d'une façon révolutionnaire son environnement.

Entreprise : Login Sécurité

Lieu : *199 Bureaux de la Colline, 92210 Saint-Cloud*

Responsable : Jean-Christophe PILLET

Mots clés (cf Thésaurus)

- Recherche fondamentale
- Transport et Télécommunications
- Informatique
- Produits chimiques



Remerciements

Qu'est que c'est?. C'est une phrase français avant le lorem ipsum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Qu'est que c'est?. C'est une phrase français avant le lorem ipsum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Qu'est que c'est?. C'est une phrase français avant le lorem ipsum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Sommaire

Remerciements

Liste des tableaux

Table des figures

Introduction	1
1 État de l’art de la menace logicielle et mécanismes d’évasion	2
1.1 Définition et taxonomie du malware moderne	2
1.2 Classification par objectifs opérationnels	3
1.3 Mécanismes d’obfuscation et d’auto-édition	4
1.4 L’émergence de la génération PromptFlux : une rupture paradigmatique	5
1.5 État actuel de la détection et ses limites	6
2 Étude de cas : analyse de PromptLock	8
2.1 Présentation du cas	8
2.2 Analyse technique de l’échantillon	9
2.3 Outils de détection développés	9
Conclusion et perspectives	11
Bibliographie	
A Annexes	I
A.1 Captures d’écran	I
A.2 Formules chimiques	I
A.3 Extraits de code source	I

Liste des tableaux

1.1	Principales techniques d'auto-édition	5
1.2	Comparaison : métamorphisme classique vs génération PromptFlux	6

Table des figures

Introduction

L'année 2025 aura marqué un tournant dans l'histoire de la cybersécurité : celui où l'intelligence artificielle générative a cessé d'être un simple outil au service de l'attaquant utilisé pour la rédaction de courriels de *phishing* et comme une assistance au développement de code malveillant pour devenir un composant actif, intégré au cœur même de l'exécution du *malware*. Cette bascule, documentée par le Google Threat Intelligence Group (GTIG) dans son rapport *AI Threat Tracker* de novembre 2025, invite à reconsidérer en profondeur les paradigmes de détection sur lesquels reposent aujourd'hui les Security Operations Centers.

C'est dans ce contexte que s'inscrit le présent mémoire, rédigé au terme d'une année d'alternance effectuée au sein de Login Sécurité, *Managed Security Service Provider* (MSSP) et « étoile » du groupe Constellation, en qualité d'analyste SOC/VOC (Security Operations Center / Vulnerability Operations Center). Mes missions y consistent principalement en la surveillance et l'investigation des systèmes d'information de nos clients, au moyen d'une plateforme SIEM assurant la collecte et la corrélation des journaux d'activité. Ce travail s'organise selon un cycle rigoureux : qualification et priorisation des alertes générées par nos outils, investigation forensique approfondie, remédiation et réponse aux incidents, puis amélioration continue des processus de détection et d'analyse cette dernière étape constituant le socle même de la réflexion développée ici.

C'est en effet au cours de cette phase d'amélioration continue qu'est né le questionnement à l'origine de ce travail. L'analyse d'alertes générées par Microsoft Defender m'a confronté à une nomenclature de détection dont la structure associant type, plateforme, famille, variante et suffixe ne m'était alors pas familière. Plutôt que de considérer cette lacune comme un simple détail technique, j'ai fait le choix d'en approfondir la logique, démarche qui constitue l'objet du premier chapitre de l'état de l'art ci-après.

Cette investigation a mis en lumière un élément en apparence secondaire mais en réalité révélateur : le suffixe `!ml`, apposé par Microsoft à certains identifiants de détection pour signaler l'intervention d'un mécanisme de *machine learning*. Ce constat a suscité une interrogation plus large quant au rôle croissant de l'intelligence artificielle dans l'écosystème des cybermenaces non plus seulement comme instrument défensif, mais comme vecteur offensif à part entière. Les recherches menées sur ce sujet m'ont conduit au rapport GTIG précité, qui documente l'émergence d'une nouvelle génération de *malwares* intégrant des modèles de langage (LLM) directement dans leur cycle d'exécution. Deux exemples y sont particulièrement significatifs : PromptFlux, un *dropper* écrit en VBScript interagissant avec l'API Gemini pour réécrire dynamiquement son propre code selon une logique dite « *just-in-time* », et PromptLock, un *ransomware* capable de générer à la volée des scripts Lua de chiffrement et d'exfiltration. L'un comme l'autre rompent avec les logiques déterministes qui caractérisaient jusqu'alors le polymorphisme et le métamorphisme classiques.

Cette découverte constitue le fil conducteur de la réflexion développée dans ce mémoire : dans quelle mesure les mécanismes de détection actuels conçus pour un adversaire dont le comportement demeure, *in fine*, modélisable sont-ils en mesure de s'adapter face à des menaces capables de générer, à chaque itération, un code radicalement différent et contextuellement optimisé pour contourner les défenses en place ?

Chapitre 1

État de l'art de la menace logicielle et mécanismes d'évasion

1.1 Définition et taxonomie du malware moderne

Un malware (malicious software) ne se limite plus aujourd'hui à sa simple capacité de réplication, comme pouvaient l'être les premiers virus informatiques des années 1980. Sa définition repose désormais sur son intention malveillante : tout code, script ou programme conçu pour infiltrer un système, perturber son fonctionnement, en extraire des données sensibles ou obtenir un accès non autorisé sans le consentement de son propriétaire.

Cette évolution reflète une transformation profonde de la menace : d'un phénomène essentiellement expérimental, elle est devenue une activité structurée, souvent pilotée par des objectifs financiers, stratégiques ou géopolitiques.

1.1.1 Nomenclature et standards d'identification

Pour analyser et communiquer efficacement sur les malwares, le secteur s'appuie sur des conventions de nommage standardisées. Microsoft, comme la grande majorité des éditeurs de sécurité, adopte le schéma défini par le CARO (Computer Antivirus Research Organization), qui structure le nom d'un malware selon le format suivant :

Type : Plateforme / Famille . Variante ! Suffixe

Chaque composant a une signification précise.

Le type décrit ce que le malware fait sur la machine infectée. Microsoft distingue plusieurs grandes catégories, parmi lesquelles : **Backdoor**, **Exploit**, **Ransom**, **Trojan**, **TrojanDownloader**, **Worm**, **Virus**, ou encore **PWS** (Password Stealer). À cela s'ajoutent des catégories pour les logiciels indésirables (**Adware**, **BrowserModifier**) et les applications potentiellement indésirables (**PUAMiner**, **PUATorrent**).

La plateforme guide le malware vers son environnement d'exécution compatible. Elle peut désigner un système d'exploitation (**Win32**, **Win64**, **Linux**, **AndroidOS**, **macOS_X**), un langage de script (**JS**, **Python**, **VBS**), ou encore un format de fichier spécifique (**XML**, **HTML**). Cette précision est importante : un même malware peut exister en plusieurs versions ciblant des plateformes différentes.

La famille correspond au nom attribué lors de la découverte initiale de la menace, regroupant des échantillons partageant des caractéristiques communes souvent une attribution aux mêmes auteurs. Il est courant que différents éditeurs de sécurité utilisent des noms de famille différents pour désigner le même malware, ce qui souligne l'importance du partage de renseignements sur les menaces (Threat Intelligence).

La variante est une lettre attribuée séquentiellement à chaque version distincte d'une même famille. Par exemple, la variante `.AF` est créée après la variante `.AE`, permettant de suivre précisément l'évolution d'une menace dans le temps.

Le suffixe, précédé d'un point d'exclamation (!), est un indicateur interne qualifiant une caractéristique technique particulière de l'échantillon : usage d'un packer, mécanisme d'obfuscation spécifique, ou composant de machine learning (!ml).

À titre d'exemple, l'identifiant `Trojan:Win32/Emotet.AF!ml` se lit ainsi : un trojan ciblant les systèmes Windows 32-bit, appartenant à la famille Emotet, dans sa variante AF, détecté via un mécanisme de machine learning.

1.1.2 Distinction fondamentale : tactique, technique, vecteur et charge utile

Avant de classer les menaces, il est essentiel de distinguer plusieurs notions souvent confondues une confusion qui fausse l'analyse de risque. Le framework MITRE ATT&CK, référence internationale pour l'analyse des comportements malveillants, propose une grille de lecture structurée en tactiques et techniques.

La tactique représente l'objectif stratégique de l'attaquant à une étape donnée de l'intrusion. Par exemple, Initial Access (TA0001) désigne la tactique consistant à pénétrer dans un système cible.

La technique décrit comment l'attaquant réalise cet objectif. Par exemple, le Phishing (T1566) est la technique utilisée pour obtenir un accès initial, l'email en étant le vecteur concret d'exécution. De même, l'exploitation d'une vulnérabilité exposée sur internet (T1190) est une autre technique relevant de cette même tactique.

Le vecteur est le support physique ou logique par lequel la technique est mise en oeuvre : un email, une clé USB, une page web compromise.

La fonctionnalité (Capability) regroupe les outils dont dispose le malware pour accomplir sa mission : enregistrement de frappes clavier (keylogging), capture d'écran, mouvement latéral dans le réseau.

La charge utile (Payload) est l'action finale recherchée par l'attaquant : chiffrement des données pour un ransomware, exfiltration de mots de passe pour un infostealer.

Le mécanisme d'évasion (Defense Evasion) recouvre toutes les techniques permettant au malware de rester invisible : obfuscation du code (T1027), détection de sandboxes, désactivation des outils de sécurité.

Cette grille de lecture permet de raisonner non plus sur ce qu'est un malware, mais sur ce qu'il fait ce qui est fondamental pour concevoir des défenses adaptées.

1.2 Classification par objectifs opérationnels

Les malwares peuvent être classés selon les objectifs poursuivis par leurs auteurs. Cette approche fonctionnelle est plus utile en pratique qu'une simple taxonomie par type de fichier.

Parmi les grandes familles, le ransomware chiffre les données de la victime et exige une rançon pour les restituer. C'est aujourd'hui l'un des modèles économiques dominants de la cybercriminalité organisée. L'infostealer est conçu pour extraire discrètement des informations sensibles identifiants, cookies de session, portefeuilles de cryptomonnaies en restant aussi silencieux que possible. Le RAT (Remote Access Trojan) permet une prise de contrôle complète à distance, facilitant l'espionnage, la surveillance ou le déplacement latéral au sein d'un réseau d'entreprise. Enfin, le loader ou dropper joue un rôle d'infrastructure dans la chaîne d'attaque : sa seule mission est de télécharger, installer et maintenir d'autres charges malveillantes plus complexes.

On distingue également plusieurs sous-catégories notables. Le spyware surveille les activités de l'utilisateur à son insu (keylogging, capture d'écran, collecte de données de navigation) et est utilisé dans des contextes d'espionnage industriel ou de surveillance ciblée. Le banker, ou trojan bancaire, est spécialisé dans le vol d'informations financières : il peut intercepter des identifiants bancaires ou injecter du contenu malveillant directement dans les pages web (web injects). Le bot intègre la machine compromise à un réseau contrôlé à distance (botnet), utilisé pour lancer des attaques DDoS, envoyer du spam ou miner des cryptomonnaies. L'exploit tire parti d'une vulnérabilité spécifique d'un logiciel pour y exécuter du code arbitraire et sert généralement de point d'entrée dans une chaîne d'infection plus large.

Ces catégories ne doivent pas être perçues comme étanches. En pratique, un même échantillon combine fréquemment plusieurs fonctionnalités un trojan bancaire intégrant des capacités d'évasion et de propagation réseau, par exemple. C'est précisément cette polyvalence qui complexifie les stratégies de détection modernes.

1.3 Mécanismes d'obfuscation et d'auto-édition

L'un des défis majeurs du reverse engineering est d'analyser un code qui change de forme de manière autonome. On parle d'auto-édition pour désigner l'ensemble des techniques permettant à un malware de modifier sa propre structure afin d'échapper à la détection, qu'elle soit statique (lecture du code sans l'exécuter) ou dynamique (observation du comportement en temps réel).

1.3.1 *Du polymorphisme au métamorphisme*

Le polymorphisme est la technique la plus répandue : le malware chiffre son corps principal et change sa clé de déchiffrement à chaque nouvelle exécution. Le contenu malveillant reste identique en mémoire, mais son empreinte sur disque est différente à chaque fois. Des familles comme Virut ou Virlock illustrent bien cette approche.

Le métamorphisme va beaucoup plus loin : le malware réécrit entièrement son propre code à chaque infection, en substituant des instructions équivalentes (`ADD EAX, 1` → `INC EAX`), en permutant les registres ou en dispersant le code en fragments reliés par des sauts. Analyser un échantillon de Zmist l'un des métamorphes les plus avancés revient à tenter de stabiliser du mercure : la logique de contrôle de flux change à chaque infection, rendant la création de signatures génériques extrêmement difficile.

1.3.2 Virtualisation de code

Des outils comme VMProtect, et certains malwares sur mesure, poussent l’obfuscation encore plus loin via la virtualisation de code : les instructions CPU classiques (x86/x64) sont transformées en un bytecode propriétaire, exécuté par une mini machine virtuelle intégrée au malware. Lors de l’analyse, on ne voit plus les instructions habituelles (`MOV`, `PUSH`, `CALL`), mais une boucle d’interprétation opaque. L’analyste doit alors reconstruire ce bytecode opération appelée *lifting* pour comprendre la logique sous-jacente.

1.3.3 Prédicats opaques et *junk code*

Une autre technique courante consiste à insérer des prédicats opaques : des structures conditionnelles (`if/else`) dont le résultat est connu à l’avance par l’attaquant, mais indéchiffrable pour un désassembleur automatique. Cela force l’outil d’analyse à explorer des chemins d’exécution qui n’existent pas en pratique (code mort), rendant le graphe de flot de contrôle (CFG) totalement illisible.

Technique	Principe	Exemples
Polymorphisme	Chiffrement du corps, clé variable à chaque exécution	Virut, Storm Worm, Virlock
Métamorphisme	Réécriture complète du code à chaque infection	Zmist, Simile, Win32/Expiro
Virtualisation	Conversion en bytecode propriétaire + VM intégrée	VMProtect, malwares custom
Prédicats opaques	Conditions insolubles pour les désassembleurs	Courant dans les packers avancés
Obfuscation dynamique	Décompression multi-étapes en mémoire	Emotet, TrickBot
Génération par IA	Réécriture sémantique via LLM	PromptFlux, Quiet-Vault

TABLE 1.1 – Principales techniques d’auto-édition

1.4 L’émergence de la génération PromptFlux : une rupture paradigmatique

L’apparition de malwares exploitant des modèles de langage (LLM) marque une rupture majeure dans l’histoire des cybermenaces. La génération dite PromptFlux en est l’illustration la plus avancée.

1.4.1 Ce qui distingue PromptFlux des métamorphes classiques

Un moteur métamorphique traditionnel repose sur des règles mathématiques finies et déterministes : une fois l’algorithme de mutation compris, il devient possible de créer une signature générique capable de détecter toutes ses variantes. La limite est donc mathématiquement contournable.

PromptFlux rompt avec cette logique. Plutôt que de permuter des instructions selon des règles prédéfinies, il utilise un modèle de langage génératif via API distante ou modèle embarqué localement pour rédiger ses fonctions en temps réel. Le résultat est radicalement différent à chaque itération, non plus syntaxiquement mais sémantiquement : la logique de la fonction peut être entièrement repensée à chaque exécution.

Critère	Métamorphisme classique	Génération PromptFlux
Logique	Algorithmique / Déterministe	Sémantique / Probabiliste
Structure	Identique (même graphe déformé)	Radicalement différente à chaque itération
Adaptabilité	Nulle face au contexte	Réécriture ciblée selon l'EDR détecté
Détection	Signature heuristique possible	Nécessite une analyse comportementale (IA contre IA)

TABLE 1.2 – Comparaison : métamorphisme classique vs génération PromptFlux

1.4.2 L'adaptation contextuelle

La capacité la plus redoutable de PromptFlux n'est pas simplement de changer de forme, mais de s'adapter à son environnement. En analysant les outils de sécurité présents sur la machine cible (version de l'EDR, politique de sécurité, système d'exploitation), il peut générer un code spécifiquement optimisé pour contourner les défenses en place à la manière d'un serrurier qui fabriquerait une clé sur mesure après avoir observé la serrure.

1.4.3 L'asymétrie économique

Un facteur aggravant souvent sous-estimé est l'asymétrie du coût : produire une nouvelle mutation via un LLM coûte quelques centimes à l'attaquant, tandis que l'analyser demande plusieurs heures de travail à un expert hautement qualifié. Cette inversion du rapport coût/effort change profondément l'économie de la cybermenace.

1.5 État actuel de la détection et ses limites

Face à ces menaces, les solutions de cybersécurité reposent sur trois approches complémentaires, chacune présentant des angles morts face aux techniques modernes.

La détection par signatures est basée sur des empreintes cryptographiques (SHA-256) des fichiers connus. Elle reste efficace pour identifier des malwares déjà répertoriés, mais est totalement inopérante face au polymorphisme ou à la génération par IA.

L'analyse heuristique tente de détecter des structures de code suspectes ou des appels système caractéristiques, sans se fier à une empreinte précise. Elle représente une avancée, mais reste contournable par des techniques d'obfuscation suffisamment sophistiquées.

L'analyse comportementale (EDR/XDR) observe les actions du programme en temps réel : tentative de chiffrement massif de fichiers, injection de code dans des processus légitimes, communications réseau anormales. C'est l'approche la plus robuste, mais elle se heurte à un

problème fondamental face à PromptFlux : si le malware change subtilement et en permanence sa manière de se comporter en imitant les patterns de mises à jour logicielles légitimes par exemple il se fond progressivement dans le bruit de fond, rendant la détection par dérive (drift analysis) de plus en plus difficile.

La conclusion de cet état de l'art est sans appel : les défenses actuelles ont été conçues pour un adversaire dont le comportement est modélisable. L'introduction de la génération sémantique par LLM introduit une plasticité fondamentalement nouvelle, qui appelle des réponses défensives tout aussi innovantes notamment l'opposition IA contre IA.

Chapitre 2

Étude de cas : analyse de PromptLock

2.1 Présentation du cas

2.1.1 Contexte et découverte

Un mois avant la découverte de PromptLock, le CERT-UA (Computer Emergency Response Team d'Ukraine) avait signalé *LameHug*, un malware attribué au groupe APT28 et propulsé par un LLM, capable de générer à la volée des commandes Windows malveillantes via l'API Hugging Face. Ce premier cas, bien que limité dans sa sophistication, annonçait déjà la bascule documentée quelques semaines plus tard par un cas autrement plus abouti.

Le 27 août 2025, les chercheurs d'ESET annoncent avoir découvert PromptLock, présenté comme le premier *ransomware* propulsé par l'intelligence artificielle générative jamais observé. L'échantillon, classifié `Filecoder.PromptLock.A`, a été repéré sur la plateforme d'analyse VirusTotal, où deux variantes — l'une pour Windows, l'autre pour Linux — avaient été téléversées. Anton Cherepanov, chercheur à l'origine de la découverte, souligne d'emblée la portée de cette évolution : l'intelligence artificielle, en abaissant la barrière technique nécessaire à la conception de menaces sophistiquées, rend accessible à un attaquant isolé ce qui nécessitait auparavant une équipe de développeurs expérimentés.

Quelques jours plus tard, le 3 septembre 2025, ESET est contacté par les auteurs d'une étude académique intitulée *Ransomware 3.0 : Self-Composing and LLM-Orchestrated*, menée par une équipe de doctorants et post-doctorants de la NYU Tandon School of Engineering. Le prototype de recherche qu'ils décrivent présente des similitudes frappantes avec les échantillons découverts sur VirusTotal, ce qui conduit ESET à revoir son interprétation initiale : PromptLock ne serait pas un outil opérationnel déployé par un groupe cybercriminel, mais très probablement le fruit de ce projet académique, destiné à illustrer les risques posés par cette nouvelle classe de malwares. ESET maintient toutefois la validité de sa découverte technique, PromptLock demeurant le premier cas connu de ransomware piloté par IA, qu'il s'agisse ou non d'une menace active.

PromptLock sera par la suite mentionné, aux côtés d'autres familles de malwares assistés par IA telles que PromptFlux ou PromptSteal, dans le rapport de synthèse *AI Threat Tracker* publié par le Google Threat Intelligence Group (GTIG) en novembre 2025 — rapport qui documente, pour sa part, la découverte de menaces distinctes intégrant des LLM directement dans leur boucle d'exécution.

C'est sur l'échantillon documenté par ESET (SHA-256 : 09bf891b7b35b2081d3ebca8de715da07a701512) que porte l'analyse développée dans ce chapitre.

2.1.2 Composition et architecture du malware

2.2 Analyse technique de l'échantillon

2.2.1 Analyse statique

2.2.2 Analyse dynamique

2.3 Outils de détection développés

2.3.1 Règle YARA

2.3.2 Plugin Volatility3

Dans le cadre d'un projet de groupe mené au sein de nos cours, nous avons développé un plugin pour le framework **Volatility 3**, orienté vers la détection de traces d'intelligences artificielles locales en mémoire vive. Ce projet a été l'occasion de faire converger deux axes de travail : d'une part les enseignements dispensés en analyse forensique mémoire, et d'autre part les travaux menés dans le cadre de ma thèse portant sur l'analyse du ransomware **PromptLock**, qui exploite des modèles de langage locaux pour générer dynamiquement une partie de son comportement malveillant.

Ce rapprochement a permis de concevoir un outil, nommé **LlmFinder**, répondant à un besoin concret identifié durant l'analyse de PromptLock : la difficulté, en réponse à incident, de mettre en évidence qu'un LLM local (Ollama, LM Studio, GPT4All, Jan, llama.cpp, etc.) a été utilisé sur une machine compromise, alors même que ce type d'usage laisse peu de traces disque persistantes et n'est pas couvert par les plugins Volatility 3 existants.

L'émergence de menaces s'appuyant sur des modèles de langage exécutés localement, à l'image de PromptLock, change la donne pour l'investigation numérique : la logique malveillante n'est plus figée dans un binaire mais générée à la volée par un LLM interrogé en local, ce qui limite fortement l'intérêt des approches classiques d'analyse statique et de détection par signature.

Dans ce contexte, la mémoire vive devient une source de preuve particulièrement précieuse, car elle peut conserver, au moment de l'acquisition, des éléments qui n'existent jamais sous forme persistante sur le disque : le prompt envoyé au modèle, la réponse générée, l'endpoint local interrogé, ou encore le processus ayant servi d'interface avec le LLM. LlmFinder a été conçu pour exploiter cette fenêtre d'observation et fournir à l'analyste DFIR des indices exploitables rapidement lors d'une investigation.

Le plugin structure ses résultats autour de quatre axes complémentaires, pensés pour couvrir l'ensemble de la chaîne d'usage d'un LLM local sur un système compromis :

- **Processus IA** : identification des processus associés à des applications d'IA locale connues (Ollama, LM Studio, GPT4All, Jan, llama.cpp), permettant de confirmer la présence effective d'un moteur de génération sur la machine analysée ;
- **Modèles** : détection de chemins de modèles présents en mémoire (répertoires `.ollama\models`, LM Studio, Jan, GPT4All, Hugging Face), donnant une indication sur le ou les modèles réellement chargés et utilisés ;
- **Réseau** : mise en évidence des endpoints locaux liés aux API d'IA (API Ollama, API

compatibles OpenAI), révélant les communications internes entre un processus tiers et le moteur LLM, souvent caractéristiques d’une utilisation programmatique plutôt qu’interactive ;

- **Prompts** : extraction de prompts probables, qu’ils aient été soumis via une interface graphique, une API locale ou un terminal, ce qui constitue l’élément de preuve le plus directement exploitable pour reconstituer l’intention et le comportement du code malveillant.

L’articulation de ces quatre familles d’informations permet de reconstituer, à partir d’une simple image mémoire, une chaîne cohérente : quel processus a sollicité quel modèle, via quel canal, avec quel contenu. Cette approche transversale est particulièrement adaptée à l’analyse de menaces de type PromptLock, où la preuve de compromission ne réside pas dans un artefact statique mais dans l’interaction dynamique entre un binaire et un LLM local.

LlmFinder fournit des indices forensiques et non des conclusions automatiques. Il ne doit pas être utilisé seul pour affirmer qu’une action malveillante a eu lieu : les résultats doivent systématiquement être corrélés avec d’autres artefacts (timeline, processus, fichiers, connexions réseau, logs) et replacés dans le contexte global de l’investigation.

2.3.3 Indices de Compromissions

Conclusion et perspectives

Qu'est que c'est?. C'est une phrase français avant le lorem ipsum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Qu'est que c'est?. C'est une phrase français avant le lorem ipsum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliographie

- [1] ARISTOTE. *La Politique*. 300 AEC.
- [2] Aurélien BARRAU. *Quand la Science appelle à l'aide pour l'humanité*. ([short.url](#)). Thinkerview. 14 sept. 2018.
- [3] Mickael CORREIA. « Loi "climat" : les pauvres et la planète attendront ». In : *Mediapart* (22 avr. 2021). ([short.url](#)).
- [4] Paul Adrien Maurice DIRAC. *The Principles of Quantum Mechanics*. International series of monographs on physics. Clarendon Press, 1981. ISBN : 9780198520115.
- [5] Pauline GUIRAGOSSIAN. « Former le citoyen-soldat sous la République jacobine ». In : *L'éducation des citoyens, l'éducation des gouvernants*. Aix-en-Provence, France, sept. 2019. URL : <https://hal-amu.archives-ouvertes.fr/hal-02115427>.
- [6] James W. KUROSE et Keith W. ROSS. *Computer Networking : A Top-Down Approach*. 8^e éd. url : ([short.url](#)). Pearson, 2021.
- [7] Tancrède RAMONET. *Ni Dieu ni maître, une histoire de l'anarchisme*. 1 :20 :48 ([short.url](#)) - Editorial Moscou. ARTE. 2019.
- [8] Pablo SERVIGNE et Gauthier CHAPELLE. *L'entraide, l'autre loi de la jungle*. Les Liens qui Libèrent, 2019.
- [9] WIKIPÉDIA. *Portail de Cryptologie*. [En ligne ; page disponible]. URL : <https://fr.wikipedia.org/wiki/Portail:Cryptologie>.

Chapitre A

Annexes

A.1 Captures d'écran

A.2 Formules chimiques

A.3 Extraits de code source

Table des matières

Remerciements

Liste des tableaux

Table des figures

Introduction	1
1 État de l'art de la menace logicielle et mécanismes d'évasion	2
1.1 Définition et taxonomie du malware moderne	2
1.1.1 Nomenclature et standards d'identification	2
1.1.2 Distinction fondamentale : tactique, technique, vecteur et charge utile . .	3
1.2 Classification par objectifs opérationnels	3
1.3 Mécanismes d'obfuscation et d'auto-édition	4
1.3.1 Du polymorphisme au métamorphisme	4
1.3.2 Virtualisation de code	5
1.3.3 Prédicats opaques et junk code	5
1.4 L'émergence de la génération PromptFlux : une rupture paradigmatique	5
1.4.1 Ce qui distingue PromptFlux des métamorphes classiques	5
1.4.2 L'adaptation contextuelle	6
1.4.3 L'asymétrie économique	6
1.5 État actuel de la détection et ses limites	6
2 Étude de cas : analyse de PromptLock	8
2.1 Présentation du cas	8
2.1.1 Contexte et découverte	8
2.1.2 Composition et architecture du malware	8
2.2 Analyse technique de l'échantillon	9
2.2.1 Analyse statique	9
2.2.2 Analyse dynamique	9
2.3 Outils de détection développés	9
2.3.1 Règle YARA	9
2.3.2 Plugin Volatility3	9
2.3.3 Indices de Compromissions	10
Conclusion et perspectives	11
Bibliographie	
A Annexes	I
A.1 Captures d'écran	I
A.2 Formules chimiques	I
A.3 Extraits de code source	I

